

SE-MSB: End-to-End Unpaired Speech Enhancement using Mamba Schrödinger Bridges

Andreas Bagge^{*†}, Andreas Nymand^{*†}, Michael Riis Andersen^{*} and Bjørn Sand Jensen^{*}

^{*}DTU Compute, Kgs. Lyngby, Denmark

[†]WSA, Lyngby, Denmark

{andreas.bagge, andreas.nymand}@wsa.com, {miri, bjj}@dtu.dk

Abstract

Speech enhancement (SE) models typically rely on supervised learning with paired data examples where clean speech is synthetically degraded. This paradigm limits performance in real-world scenarios where the target environment’s specific acoustic characteristics are unknown. We propose a fully unpaired SE framework that uses principled Diffusion Schrödinger Bridges (DSB) to learn a stochastic transport process between a clean and a degraded speech distribution. Algorithms for learning transport maps are computationally heavy since they require simulating differential equations during training, usually at each training step. Therefore, we propose using a high-efficiency Mamba Diffusion Model designed for end-to-end waveform processing. We compare against state-of-the-art methods for speech enhancement, both paired and unpaired, as well as a classical signal processing algorithm. Experimental results show that we are on par or better than the baselines while being orders of magnitude faster during inference. Furthermore, we show that the flexibility of the DSB formulation allows our model to generalize across SE tasks, offering a robust and efficient solution for real-world speech restoration.

1 Introduction

Speech enhancement (SE) problems, such as denoising, declipping, or dereverberation, are classic problems in audio processing. Traditionally, these are solved using supervised methods trained on collections of paired instances of clean speech x_0 and degraded speech x_1 , where the degraded speech is typically simulated by modifying the clean speech, e.g., by adding noise or clipping the amplitude such that $x_1 \sim p_{\text{noisy}}(\cdot|x_0)$. Such approaches are limited to training on simulated distortions and cannot be applied to actual, in-the-wild degraded audio because large-scale paired data collection is infeasible or, in some instances, even impossible. Enabling model training on unpaired in-the-wild audio, i.e., where clean and degraded speech are drawn from separate distributions $x_0 \sim p_{\text{clean}}, x_1 \sim p_{\text{noisy}}$ independently rather than paired samples $(x_0, x_1) \sim p_{\text{joint}}$, has the potential to improve SE models in real-world conditions by adapting the model to out-of-distribution (OOD) data, for example, by personalizing the model to the user’s specific acoustic environment. With the recent rise in personalized AI, especially through federated learning [1], the ability to train models directly on observed, but unpaired or unlabeled, data is becoming increasingly important for adapting models to the specific needs of end-users. This is especially true for SE models, where the end-user’s acoustic environment is often unknown and can be highly variable and unpredictable. In some instances, the degradation is not only unknown but also difficult to simulate to be used in a paired setup, e.g., Lombard effect [2] or reconstructing historical records [3]. Diffusion models constitute the state-of-the-art for image and audio synthesis [4, 5], and they have also shown great results for paired SE [6]. Diffusion models generally consist of two processes, a forward and a backward process. In traditional diffusion models, the forward process gradually turns data samples into Gaussian noise, and the objective of the model is to learn the backward process by effectively removing noise from

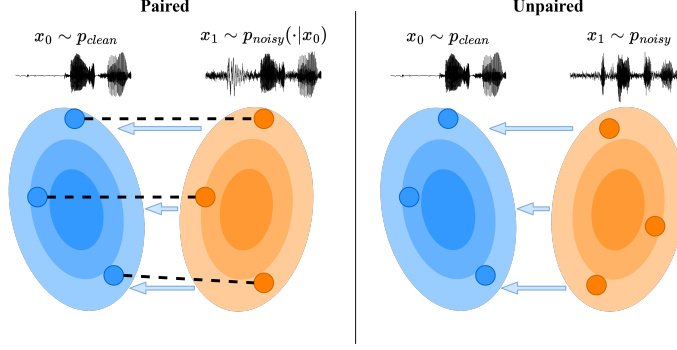


Figure 1: In traditional supervised speech enhancement (**left**), we assume access to a clean sample x_0 from the **clean target distribution** and a corresponding degraded sample from the **degraded distribution** to learn the optimal pairing (blue arrows). In contrast, our method (**right**) requires only independent samples from both distributions to learn the optimal pairing.

the input. After learning the reverse process, the models can then synthesize high-quality samples from Gaussian noise. Though traditional diffusion methods yield high-quality samples, they do not allow for mapping between two arbitrary data distributions. In contrast, Schrödinger Bridges (SB) [7] allow for high-quality sample generation and direct transfer between two arbitrary data distributions through the entropy-regularized optimal transport problem, also called the SB problem, of finding the most likely random evolution between two continuous probability distributions. That is, SBs allow learning stochastic maps between clean and degraded audio from unpaired data only, enabling training on in-the-wild data. SBs are formulated in terms of stochastic differential equations (SDEs), which can be simulated with SDE solvers, and solving the SB problem therefore allows for a diffusion-based generative approach. Mamba is a selective state-space model that serves as an efficient alternative to the popular transformer architecture for long sequence modelling [8]. Mamba offers linear scaling in sequence length and contains a constant state size during inference. We propose the SE-MSB (Speech Enhancement Mamba Schrödinger Bridges) model, which is a novel combination of Diffusion Schrödinger Bridges (DSB) and modern state-space models for end-to-end SE using completely unpaired data. We provide a fast and efficient implementation of the DSB method for speech enhancement on raw audio waveforms using a bespoke Mamba Diffusion model. Raw waveforms suffer from a higher dimensionality, which is a problem solved by Mambas linear sequence length scaling. We show that the combination of DSB and Mamba outperforms or is on par with other state-of-the-art methods for unpaired SE, while being orders of magnitude faster during inference, both due to the efficiency of the network architecture and due to the ability of the model to do few-step diffusion without sacrificing performance. Audio examples and code are available at anonymous.4open.science/r/Latent-DSB. Our main contributions are:

- We propose the SE-MSB model, which is the **first fully unpaired and end-to-end model** for speech enhancement on raw waveforms.
- We evaluate SE-MSB using a rigorous experimental protocol and compare it against several state-of-the-art baselines on **multiple tasks and compounded conditions**.
- We demonstrate that SE-MSB achieves **strong speech enhancement performance**, outperforming paired methods in some cases while exhibiting **highly competitive efficiency**.

2 Related Work

A²ASB (Audio-to-Audio Schrödinger Bridge) [9] is a tractable, simulation-free method for learning maps between the distributions $p_{data}(\mathbf{X}_0)$ and $p_{prior}(\mathbf{X}_1|\mathbf{X}_0)$, i.e., a conditional probability distribution that requires paired samples. A²ASB exhibits state-of-the-art results on paired audio restoration tasks such as bandwidth extension and inpainting. CycleGAN is a discriminative generative model for learning a deterministic mapping between distributions [10]. CycleGANs have also been used for style transfer in the context of audio, for example, for whisper-to-normal speech conversion [11]. However, CycleGANs are highly unstable during training and require jointly optimizing 4 distinct neural networks.

[12] proposes using Gaussian Flow Bridges for unpaired speech enhancement, a diffusion-based approach where samples are first mapped to an intermediate Gaussian distribution. The approach suffers from the fact that the simulated trajectories first have to map to a third distribution, a Gaussian, therefore yielding non-optimal transport. A similar class of methods for unpaired speech enhancement uses

	Unpaired	End-to-End	Task-flexible
SE-MSB	✓	✓	✓
GFB	✓	✗*	✓
BUDDy	✓	✗	✗
A ² ASB	✗	✗	✓

Table 1: Comparison of current paired/unpaired diffusion models for speech enhancement. *GFB processes raw audio but employs an internal STFT encoder and ISTFT decoder.

diffusion posterior sampling to solve the audio inverse problem in an unsupervised manner [13, 14, 15]. Instead of training a task-specific model, these methods utilize a pretrained diffusion model as a prior over the clean data. During inference, the models then modify the reverse diffusion process to guide the sampling trajectory. However, these methods also require access to the mathematical structure of the degradation process, which, in most cases, is not available for in-the-wild data. Diffusion Schrödinger Bridge Matching [16, 17] is an algorithm for computing the Schrödinger Bridge, a dynamic entropy-regularised version of optimal transport. The algorithm can be used for learning a diffusion model for mapping between arbitrary distributions using unpaired data samples. Mamba, a selective state-space model architecture, has shown great results in the field of paired speech enhancement [18] [19]. In combination with diffusion Schrödinger bridges, Mamba has shown to outperform Long Short-Term Memory and Multi-Head Self-Attention backbones for Short Time Fourier Transform (STFT) based audio representations for paired speech enhancement. However, STFT-based methods are limited by fixed transformation hyperparameters, reducing the model’s ability to capture the expressivity of raw audio.

3 Method

Our work builds upon the Diffusion Schrödinger Bridge Matching algorithm [16, 17], DSB for short. Consider two distinct continuous probability distributions on $\mathbb{R}^n$. In the literature, these distributions are usually denoted as p_{data} and p_{prior} . However, for our application of speech enhancement, we will denote these distributions as p_{clean} and p_{noisy} , respectively, to emphasize their role as the target and source distributions in the speech enhancement problem. Therefore, p_{clean} refers to the distribution of clean speech, while p_{noisy} refers to the distribution of degraded speech, for example, noisy, reverberant, or clipped speech. We will denote samples from p_{clean} and p_{noisy} as $\mathbf{X}_0$ and $\mathbf{X}_1$, respectively, i.e. $\mathbf{X}_0 \sim p_{clean}$ and $\mathbf{X}_1 \sim p_{noisy}$, where $\mathbf{X}_0, \mathbf{X}_1 \in \mathbb{R}^n$. We seek a stochastic process, or a ‘bridge’, that transports samples from p_{clean} to p_{noisy} and vice versa. We will parameterize the processes as two distinct SDEs, one for each direction, such that simulating the SDEs with a sample as the initial value will transport said sample from one domain to the other. We will denote the forward and backward SDEs as:

$$\begin{aligned} d\mathbf{X}_t &= f(\mathbf{X}_t, t)dt + g(t)d\mathbf{W}_t, & \mathbf{X}_0 &\sim p_{clean}, \\ d\mathbf{X}_t &= b(\mathbf{X}_t, t)dt + g(t)d\bar{\mathbf{W}}_t, & \mathbf{X}_1 &\sim p_{noisy}, \end{aligned}$$

where f and b are the drift terms, $g(t)$ is the diffusion term, and $\mathbf{W}$ and $\bar{\mathbf{W}}$ are standard Wiener processes. For simplicity, we will assume $g(t) = \beta$ is constant. Therefore, the problem reduces to learning the drift terms for each process, i.e., f and b , such that simulating the backward process from $t = 1$ to $t = 0$ starting from $\mathbf{X}_1 \sim p_{noisy}$ yields $\mathbf{X}_0 \sim p_{clean}$ at time $t = 0$ (and vice versa for the forward process). We will parameterize the forward and backward drift with a single neural network $v(\mathbf{X}_t, t, s)$ with parameters θ such that:

$$f(\mathbf{X}_t, t) \approx v_\theta(\mathbf{X}_t, t, 1), \quad b(\mathbf{X}_t, t) \approx v_\theta(\mathbf{X}_t, t, 0),$$

where $s \in \{0, 1\}$ is a binary indicator variable, indicating whether the forward or backward drift is being approximated. Alternatively, one could have parameterized each drift with a separate neural network. Having obtained the optimal parameters θ^* , we can approximately simulate the forward and backward SDEs on the discrete time interval $0 = t_0 < t_1 \dots < t_N = 1$, where $\Delta t_{k+1} = t_{k+1} - t_k$, using the Euler-Maruyama method:

$$p(\mathbf{X}_{t_{k+1}} | \mathbf{X}_{t_k}) \approx p(\mathbf{X}_{t_{k+1}} | \mathbf{X}_{t_k}, \mathbf{X}_1) = \mathcal{N}(\mathbf{X}_{t_{k+1}}; \tilde{\mu}_k, \tilde{\sigma}_k^2 \mathbf{I}), \quad (\text{forward process}) \quad (1)$$

$$p(\mathbf{X}_{t_k}|\mathbf{X}_{t_{k+1}}) \approx p(\mathbf{X}_{t_k}|\mathbf{X}_{t_{k+1}}, \mathbf{X}_0) = \mathcal{N}(\mathbf{X}_{t_k}; \mu_{k+1}, \sigma_{k+1}^2 \mathbf{I}). \quad (\text{backward process}) \quad (2)$$

That is, the forward and backward processes can be simulated by sampling from the respective Gaussian distributions, where the means and variances are given by:

$$\begin{aligned} \tilde{\mu}_k &\approx \mathbf{X}_{t_k} + \Delta t_{k+1} \underbrace{(\mathbf{X}_1 - \mathbf{X}_{t_k})/(1 - t_k)}_{=f(\mathbf{X}_{t_k}, t_k)} \approx \mathbf{X}_{t_k} + \Delta t_{k+1} v_\theta(\mathbf{X}_{t_k}, t_k, 1), \\ \mu_{k+1} &\approx \mathbf{X}_{t_{k+1}} + \Delta t_{k+1} \underbrace{(\mathbf{X}_0 - \mathbf{X}_{t_{k+1}})/t_{k+1}}_{=b(\mathbf{X}_{t_{k+1}}, t_{k+1})} \approx \mathbf{X}_{t_{k+1}} + \Delta t_{k+1} v_\theta(\mathbf{X}_{t_{k+1}}, t_{k+1}, 0), \end{aligned}$$

$$\tilde{\sigma}_k^2 = \frac{\beta \Delta t_{k+1} (1 - t_{k+1})}{1 - t_k}, \quad \sigma_{k+1}^2 = \frac{\beta \Delta t_{k+1} t_k}{t_{k+1}}.$$

Additionally, it can be shown [16] that given $\mathbf{X}_0$ and $\mathbf{X}_1$, then intermediate points in the bridge can be sampled from the conditional distribution:

$$p(\mathbf{X}_{t_k}|\mathbf{X}_0, \mathbf{X}_1) = \mathcal{N}((1 - t_k)\mathbf{X}_0 + t_k\mathbf{X}_1, \beta t_k(1 - t_k)\mathbf{I}). \quad (3)$$

To train the neural network v_θ , we will use the DSB Matching algorithm [16, 17]. The algorithm contains two phases: pre-training and fine-tuning. During the pre-training phase, i.i.d. samples from p_{clean} and p_{noisy} are sampled independently. We will call this pair a "random coupling" (i.e., unpaired). An intermediate point, $\mathbf{X}_{t_k}$, is sampled from the conditional distribution (3), and the objective of the network is to predict the forward and backward drift at time t_k and t_{k+1} given by $(\mathbf{X}_1 - \mathbf{X}_{t_k})/(1 - t_k)$ and $(\mathbf{X}_0 - \mathbf{X}_{t_{k+1}})/t_{k+1}$, respectively. After a number of pre-training steps, the fine-tuning phase begins. Here, the initial samples $\mathbf{X}_0$ and $\mathbf{X}_1$ are once again obtained by sampling from p_{clean} and p_{noisy} , but they are no longer randomly coupled. Instead, the couplings are obtained by simulating the processes on the interval from $t = 0$ to $t = 1$ and from $t = 1$ to $t = 0$, respectively, using the current parameters θ of the model. The objective of the model is the same as in the pre-training phase. [16] shows that applying the pre-training phase followed by the fine-tuning phase will eventually yield the SB, i.e., the optimal stochastic process between p_{clean} and p_{noisy} . The DSB Matching algorithm can be seen as a stochastic version of the Reflow algorithm [20], which can be deduced from DSB Matching by setting $\beta = 0$, i.e., by removing the noise from the SDEs. For the Reflow algorithm, the authors show that the fine-tuning phase yields "straight flows", i.e. flows that transport samples from p_{clean} to p_{noisy} along straight lines. This is a desirable property for diffusion models, as it allows for few-step diffusion sampling without compromising sample quality, therefore requiring less compute during inference. Additionally, it can also be shown that A²ASB is simply a paired version of the DSB Matching algorithm that uses only the pre-training phase, and that only trains the backward process, but instead of drawing random couplings, the couplings are drawn according to the joint distribution and are therefore paired, see Appendix C.

Mamba Diffusion Model architecture

To the best of our knowledge, all existing unpaired speech enhancement models operate on spectrograms [13, 14, 15]. This is done mostly to enable the use of established convolutional neural networks, such as U-Nets [21], on the resulting time-frequency representations. However, such time-frequency representations impose a rigid, non-learnable trade-off between temporal and frequency resolution, and, in the case of magnitude representations, completely disregard the phase, therefore requiring heuristic phase reconstruction algorithms or an additional vocoder for waveform synthesis. Therefore, we propose combining DSB with modern state-space architectures to operate in an end-to-end fashion on raw audio waveforms, enabling the model to learn an internal representation of the raw data, without relying on specific choices of time-frequency representations. To this end, we use a bespoke, efficient Mamba Diffusion model architecture based on Mamba blocks [8]. This architecture uses an initial 1D convolutional layer to learn an internal latent representation of the raw audio waveform, similar to a learnable time-frequency representation. This latent is then processed by a number of Mamba Diffusion blocks. Each Mamba Diffusion block uses Adaptive LayerNorm [22] to inject the diffusion timestep t and the process-direction conditioning c . The model architecture is visualized in Figure 3, and a more detailed description of the architecture can be found in Appendix B. To compare the algorithmic efficiency of the proposed SE-MSB model to the state-of-the-art baselines, we have opted to count tera (10^{12}) floating-point operations (TFLOPS) required to evaluate a sample output. Each of the models processed a dummy audio input of $2^{15} = 32768$ samples (2.048 seconds

Algorithm 1 Training algorithm

```

1: Initialize neural network  $v_\theta$  and batch size  $2B$ 
2: Let  $i \in \{1, 2 \dots B\}$ 
3: while not converged do
4:   if Pre-training then
5:      $\mathbf{X}_0^{1:B} \stackrel{\text{iid}}{\sim} p_{\text{clean}}, \tilde{\mathbf{X}}_1^{1:B} \stackrel{\text{iid}}{\sim} p_{\text{noisy}}$ 
6:      $\mathbf{X}_1^{1:B} \stackrel{\text{iid}}{\sim} p_{\text{noisy}}, \tilde{\mathbf{X}}_0^{1:B} \stackrel{\text{iid}}{\sim} p_{\text{clean}}$ 
7:   else
8:      $\mathbf{X}_0^{1:B} \stackrel{\text{iid}}{\sim} p_{\text{clean}}, \tilde{\mathbf{X}}_1^{1:B} \stackrel{\text{iid}}{\sim} p_{\text{noisy}}$ 
9:     Sample  $\mathbf{X}_1^i$  using (1) and  $v_\theta(\cdot, \cdot, 1)$  starting from  $\mathbf{X}_0^i$ 
10:    Sample  $\tilde{\mathbf{X}}_0^i$  using (2) and  $v_\theta(\cdot, \cdot, 0)$  starting from  $\tilde{\mathbf{X}}_1^i$ 
11:   end if
12:    $t^{1:B}, \tilde{t}^{1:B} \stackrel{\text{iid}}{\sim} \mathcal{U}(0, 1)$ 
13:    $\mathbf{X}_t^i \sim \mathcal{N}((1 - t^i)\mathbf{X}_0^i + t^i\mathbf{X}_1^i, \beta t^i(1 - t^i)\mathbf{I})$ 
14:    $\tilde{\mathbf{X}}_{\tilde{t}}^i \sim \mathcal{N}((1 - \tilde{t}^i)\tilde{\mathbf{X}}_0^i + \tilde{t}^i\tilde{\mathbf{X}}_1^i, \beta \tilde{t}^i(1 - \tilde{t}^i)\mathbf{I})$ 
15:    $\mathcal{L}_b = \frac{1}{B} \sum_{i=1}^B \left\| v_\theta(\mathbf{X}_t^i, t^i, 0) - \frac{\mathbf{X}_0^i - \mathbf{X}_1^i}{t^i} \right\|$ 
16:    $\mathcal{L}_f = \frac{1}{B} \sum_{i=1}^B \left\| v_\theta(\tilde{\mathbf{X}}_{\tilde{t}}^i, \tilde{t}^i, 1) - \frac{\tilde{\mathbf{X}}_1^i - \tilde{\mathbf{X}}_0^i}{1 - \tilde{t}^i} \right\|$ 
17:   Take gradient step  $\nabla_{\theta} \frac{1}{2}(\mathcal{L}_b + \mathcal{L}_f)$ 
18: end while

```

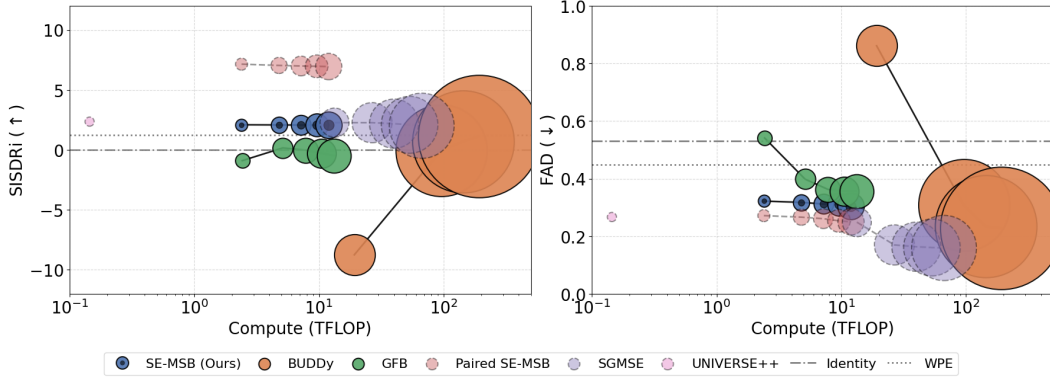


Figure 2: Total compute vs SISDRi (**left**) and FAD (**right**). The area of the points scales linearly with mean inference time, see table 3 and appendix D. Our model outperforms unpaired state-of-the-art models with the same compute budget. It is also seen that the model is robust to changes in the number of diffusion steps, performing almost the same no matter how many diffusion steps are taken. This is also shown in figure 4. The models with a dashed outline are trained on paired data. An Equivalent plot for the CHiME-6 dataset can be seen in appendix D.

of audio at 16kHz). For counting the TFLOPs of the models, PyTorch’s “FlopCounterMode” was used. As our implementation relied on the mamba-ssm library, which uses custom Triton kernels for an efficient implementation, the TFLOP count has been manually registered and is counted as explained in chapter 6 of the Mamba2 paper [8]. The majority of the Mamba2 TFLOP count comes from the 1D convolution and the selective scan algorithms, which, according to the paper, consist of an order $O(TN^2)$ FLOPs, where T is the sequence length and N is the state dimension. This is under the assumption that $N = P = Q$ where P is the head dimension and Q is the chunk size. If we assume a more general form, it can be shown that the FLOP count from the selective scan equals $FLOP_{scan} = 2BTN(HQ + 3D)$, where B is the batch size, H is the number of heads, and D is total head dimension, $D = HP$, see appendix D. Before the block decomposition, a 1D causal convolution across the inner dimension is performed with a total cost of $FLOP_{conv} = 2BTDK$, where K is the convolution kernel size. In addition to this, some elementwise operations are used in

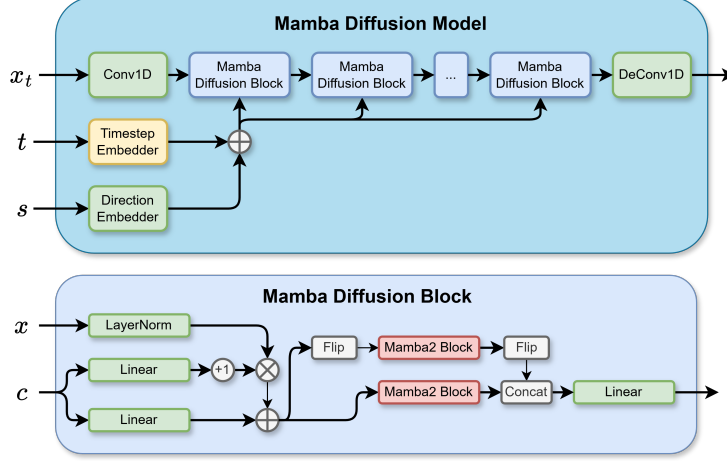


Figure 3: The Mamba Diffusion Model architecture proposed for SE-MSB. Each waveform is encoded using a 1D convolutional layer. The resulting representation is mixed with the conditional signal and processed by a stack of Mamba Diffusion Blocks.

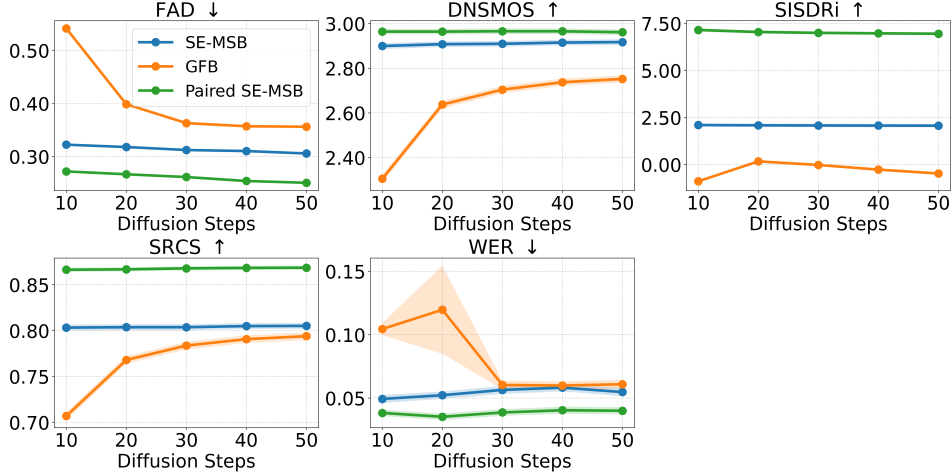


Figure 4: Metrics as a function of the number of diffusion steps. The SE-MSB model is almost unaffected by the number of steps, showing that SE-MSB can do few-step generation.

the Triton kernels, and are, as such, invisible to Torch’s “FlopCounterMode”. The FLOP count is negligible compared to the selective state scan, but for completeness, we include them in the count. First, the SiLU activation layers take approximately $5BTD$ FLOPs. Secondly, the gated RMS-Norm takes $11BTD$ FLOPs, 5 from the RMS norm, 5 for an attached SiLU, and 1 from an elementwise multiplication. Lastly, a final elementwise multiplication takes $BTHQ$ FLOPs. Further information on the FLOP count on non-torch baselines can be seen in Appendix D.

4 Experiments

The proposed SE-MSB model is trained and evaluated on a dereverberation, denoising, and declipping task. These tasks are traditionally solved with supervised methods, but by artificially degrading the clean speech data, we can evaluate distortion metrics that require paired data. The clean speech data used throughout our experiments comes from VCTK [23], see Appendix A.1. The VCTK dataset is both used as p_{clean} and to simulate p_{noisy} , but, importantly, the model never sees pairs of clean and degraded audio during training.

	Model	FAD ($\downarrow$)	DNSMOS ($\uparrow$)	SISDRi ($\uparrow$)	WER ($\downarrow$)	SR-CS ($\uparrow$)	TFLOPS ($\downarrow$)
VCTK	U SE-MSB (10)	0.32	2.90 $\pm$ 0.01	2.09 $\pm$ 0.09	0.05 $\pm$ 0.00	0.80 $\pm$ 0.00	2.3937
	U SE-MSB (50)	0.31	2.92 $\pm$ 0.01	2.06 $\pm$ 0.09	0.05 $\pm$ 0.00	0.81 $\pm$ 0.00	11.9687
	U BUDDy (10 $\dagger$)	0.86	2.30 $\pm$ 0.01	-8.75 $\pm$ 0.11	0.54 $\pm$ 0.01	0.47 $\pm$ 0.01	19.2127
	U BUDDy (100 $\dagger$)	0.23	3.08 $\pm$ 0.01	1.15 $\pm$ 0.12	0.02 $\pm$ 0.00	0.91 $\pm$ 0.00	192.1273
	U GFB (10)	0.54	2.30 $\pm$ 0.02	-0.90 $\pm$ 0.08	0.10 $\pm$ 0.00	0.71 $\pm$ 0.00	2.4265
	U GFB (50)	0.36	2.75 $\pm$ 0.02	-0.48 $\pm$ 0.04	0.06 $\pm$ 0.00	0.79 $\pm$ 0.00	13.2112
	U WPE	0.45	2.71 $\pm$ 0.02	1.24 $\pm$ 0.07	0.02 $\pm$ 0.00	0.84 $\pm$ 0.00	0.0012
	P SGMSE (10)	0.25	2.89 $\pm$ 0.02	2.27 $\pm$ 0.09	<u>0.02 $\pm$ 0.00</u>	0.90 $\pm$ 0.00	13.3338
	P SGMSE (50)	<u>0.16</u>	2.92 $\pm$ 0.02	2.01 $\pm$ 0.11	<u>0.02 $\pm$ 0.01</u>	<u>0.91 $\pm$ 0.00</u>	66.6690
	P Paired SE-MSB (10) *	0.27	2.96 $\pm$ 0.01	<u>7.15 $\pm$ 0.08</u>	0.04 $\pm$ 0.00	0.87 $\pm$ 0.00	2.3937
	P Paired SE-MSB (50) *	0.25	2.96 $\pm$ 0.01	6.95 $\pm$ 0.08	0.04 $\pm$ 0.00	0.87 $\pm$ 0.00	11.9687
	P UNIVERSE++	0.27	3.05 $\pm$ 0.01	2.35 $\pm$ 0.09	0.03 $\pm$ 0.00	0.90 $\pm$ 0.00	11.9687
	Identity	0.53	2.64 $\pm$ 0.02	0.00 $\pm$ 0.00	<u>0.02 $\pm$ 0.00</u>	0.79 $\pm$ 0.00	0.0000
	U SE-MSB (10)	0.15	2.90 $\pm$ 0.02	0.60 $\pm$ 0.09	0.11 $\pm$ 0.02	0.67 $\pm$ 0.00	2.3937
	U SE-MSB (50)	0.16	2.92 $\pm$ 0.02	0.52 $\pm$ 0.09	0.13 $\pm$ 0.01	0.66 $\pm$ 0.00	11.9687
	U BUDDy (10 $\dagger$)	0.69	2.21 $\pm$ 0.02	-9.95 $\pm$ 0.15	0.76 $\pm$ 0.02	0.38 $\pm$ 0.01	19.2127
Libri	U BUDDy (100 $\dagger$)	0.17	3.18 $\pm$ 0.01	-1.76 $\pm$ 0.18	0.04 $\pm$ 0.00	0.80 $\pm$ 0.00	192.1273
	U GFB (10)	0.31	2.59 $\pm$ 0.02	-1.14 $\pm$ 0.07	0.16 $\pm$ 0.01	0.59 $\pm$ 0.01	2.4265
	U GFB (50)	0.29	2.90 $\pm$ 0.02	-0.33 $\pm$ 0.04	0.08 $\pm$ 0.00	0.71 $\pm$ 0.01	13.2112
	U WPE	0.22	2.71 $\pm$ 0.03	1.40 $\pm$ 0.07	0.02 $\pm$ 0.00	0.89 $\pm$ 0.00	0.0012
	P SGMSE (10)	0.25	2.89 $\pm$ 0.02	2.27 $\pm$ 0.09	<u>0.02 $\pm$ 0.00</u>	0.90 $\pm$ 0.00	13.3338
	P SGMSE (50)	0.16	2.92 $\pm$ 0.02	2.01 $\pm$ 0.11	<u>0.02 $\pm$ 0.01</u>	<u>0.91 $\pm$ 0.00</u>	66.6690
	P Paired SE-MSB (10) *	0.20	3.01 $\pm$ 0.02	<u>2.75 $\pm$ 0.12</u>	0.08 $\pm$ 0.00	0.67 $\pm$ 0.01	2.3937
	P Paired SE-MSB (50) *	0.20	3.03 $\pm$ 0.02	2.72 $\pm$ 0.12	0.08 $\pm$ 0.00	0.67 $\pm$ 0.01	11.9687
	P UNIVERSE++	0.22	2.96 $\pm$ 0.02	-1.08 $\pm$ 0.17	0.18 $\pm$ 0.02	0.66 $\pm$ 0.01	11.9687
	Identity	0.28	2.57 $\pm$ 0.03	0.00 $\pm$ 0.00	0.03 $\pm$ 0.00	0.82 $\pm$ 0.00	0.0000

Table 2: Evaluation metrics for the dereverberation task with 95% confidence intervals. The Identity baseline corresponds to unprocessed audio. The number of diffusion steps is denoted in parentheses when relevant, except for $\dagger$, which denotes optimization steps. U = unpaired, P = paired. Best unpaired model in each metric is **bold**, and best model across paired and unpaired models is underlined.

The models are tested on the test split of VCTK using an unseen test set of room impulse responses. Standard supervised speech enhancement models are conventionally evaluated using isolated test sets to assess generalizability to OOD shifts. In contrast, the primary advantage of unpaired speech enhancement is the ability for in-domain training, allowing the model to train directly on the observed target distribution. Therefore, to verify that our model retains generalizability, we also evaluate all models on a separate, independent clean speech dataset, namely the Libri ASR Corpus [24].

- **Dereverberation:** The degraded speech is generated by applying room impulse responses (RIRs) from various public RIR datasets [25, 26, 27, 28, 29] to the clean speech.
- **Denoising and declipping:** The degraded speech is generated via additive noise and, in a separate experiment, clipping the resulting mixture. The noise dataset is WHAM! noise [30] mixed at 5 dB SNR. Clipping is randomly applied to remove between 0 and 1 dB of the signal power.

For each task, we train our SE-MSB model using the Mamba Diffusion Model architecture on raw audio waveforms using $\beta = 0.1$ following a parameter sweep. The model is trained for 250k steps during pre-training and 100k steps during fine-tuning for a total of 350k steps. We use a batch size of 18 on audio samples of 1.00 seconds at a sample rate of 16 kHz. During the fine-tuning phase, each SDE is simulated using 20 steps of the Euler-Maruyama method. For dereverberation, we compare the performance of the SE-MSB model against both unpaired and paired baselines. The unpaired baselines include a Gaussian Flow Bridge (GFB) method [12], a diffusion posterior sampling method called BUDDy [14], and the Weighted Prediction Error (WPE) method [31]. For the paired baselines, we evaluate against a Score-based Generative Model for Speech Enhancement (SGMSE) [6], and the supervised version of the SE-MSB method called A²SB [9]. We also include an Identity baseline corresponding to unprocessed reverberant audio. Baselines are described in detail in Appendix C.

To showcase the flexibility of our method, we train an SE-MSB model on a mixture of multiple degradations: reverberation, additive noise, and clipping. We compare the performance of the

	Model	TFLOPS	Inference Time (ms)	Trainable Params ($\times 10^6$)
U	SE-MSB (10)	2.3937	214.19 $\pm$ 0.53	45.56
U	SE-MSB (50)	11.9687	1042.29 $\pm$ 6.59	45.56
U	BUDDy (10 $\dagger$)	19.2127	2481.45 $\pm$ 8.02	27.74
U	BUDDy (100 $\dagger$)	192.1273	22135.40 $\pm$ 17.10	27.74
U	GFB (10)	2.4265	314.67 $\pm$ 0.44	43.85
U	GFB (50)	13.2112	1713.99 $\pm$ 3.99	43.85
U	WPE	0.0012	80.72 $\pm$ 0.25	0.00
P	Sepformer	0.4722	55.21 $\pm$ 0.18	25.61
P	SGMSE (10)	13.3338	1253.28 $\pm$ 1.85	65.59
P	SGMSE (50)	66.6690	6276.25 $\pm$ 5.70	65.59
P	Paired SE-MSB (10)	2.3937	215.00 $\pm$ 0.39	45.56
P	Paired SE-MSB (50)	11.9687	1031.56 $\pm$ 0.73	45.56
P	UNIVERSE++	0.1446	124.95 $\pm$ 2.36	84.24

Table 3: Computational efficiency: FLOPS, inference time, and number of trainable parameters.

SE-MSB model against BUDDy, and the unprocessed baseline. BUDDy uses a diffusion posterior sampling method, but BUDDy also assumes a specific degradation, namely a convolution with some unknown RIR, and is therefore sensitive to these assumptions being violated. For the tasks involving additive noise, we also compare against a paired baseline, namely the Sepformer [32], a state-of-the-art supervised speech enhancement model.

We test the few-step sampling capabilities of both the paired and unpaired SE-MSB model compared to the unpaired diffusion baseline GFB on the dereverberation task by evaluating metrics for different numbers of sampling steps. The number of sampling steps can be changed at inference time, and therefore, only a single trained model is required for each task. Few-step sampling is a desirable property as it can reduce the inference time of the model, and therefore increase the practicality of the method for real-world applications. Lastly, we also measure the computational efficiency of all models using FLOPS, inference time (wall clock time), and the number of trainable parameters. The generated samples are evaluated using WER (word error rate), pMOS/DNSMOS (predicted mean opinion score using the DNSMOS model) [33], SR-CS (speaker recognition cosine similarity), scale-invariant signal-to-distortion ratio (SISDRi) [34], and FAD (Fréchet Audio Distance) [35]. WER, SR-CS, and SISDRi can be interpreted as content fidelity measures capturing how well content is preserved, while pMOS and FAD can be interpreted as perceptual quality metrics. WER is calculated using the small Whisper model [36] to obtain ground-truth transcriptions of the reference (clean) speech. The transcription model is robust towards distorted speech and is therefore not necessarily a good measure of how well the audio is reconstructed. Nevertheless, WER serves as a useful indicator of whether the semantic content of the speech is preserved. SR-CS is calculated using a speaker embedding model [37] to obtain speaker embeddings of the ground truth signal and the generated signal. The metric is then calculated as the cosine similarity between these embeddings. The samples used in the FAD metric are encodings produced using a CLAP model [38].

5 Results

Results of the dereverberation task are shown in Table 2, results of the mixed degradation task are shown in Table 4, and the results of the few-step sampling experiment are shown in Figure 4. Computational efficiency is shown in Table 3. The dereverberation and computational efficiency results show that while SE-MSB is moderately outperformed by BUDDy on most metrics, SE-MSB is around two orders of magnitude more efficient than BUDDy in terms of FLOPS. This large computation difference is made possible thanks to the linear time sequence scaling of the Mamba diffusion model. Additionally, we see that while all unpaired methods are outperformed by the paired methods (SGMSE and Paired SE-MSB) on most metrics, the difference is negligible for some metrics, showing that SE-MSB can achieve performance comparable to paired methods.

The results of the mixed degradation task show that SE-MSB outperforms BUDDy by a large margin when the degradation is no longer only a convolution with a room impulse response, but also includes additive noise and clipping. SE-MMSB achieves comparable performance to the

	Model	FAD ($\downarrow$)	DNSMOS ($\uparrow$)	SISDRi ($\uparrow$)	WER ($\downarrow$)	SR-CS ($\uparrow$)
Reverb + Noise	U SE-MSB (10)	0.35	2.72 ± 0.01	4.95 ± 0.13	0.08 ± 0.00	0.74 ± 0.00
	U SE-MSB (50)	0.33	2.74 ± 0.01	4.91 ± 0.13	0.09 ± 0.00	0.75 ± 0.00
	U BUDDy (100 $\dagger$)	0.70	2.66 ± 0.01	0.73 ± 0.10	0.05 ± 0.00	0.83 ± 0.00
	P Sepformer	0.29	2.90 ± 0.01	5.00 ± 0.08	<u>0.05 ± 0.00</u>	0.83 ± 0.00
	P SGMSE+	0.30	2.80 ± 0.01	4.49 ± 0.11	<u>0.05 ± 0.01</u>	0.84 ± 0.00
	P UNIVERSE++	<u>0.27</u>	<u>3.01 ± 0.01</u>	<u>6.32 ± 0.09</u>	0.08 ± 0.00	<u>0.88 ± 0.00</u>
	Identity	1.05	1.66 ± 0.03	0.00 ± 0.00	0.04 ± 0.00	0.79 ± 0.00
Reverb + Noise + Clip	U SE-MSB (10)	0.30	2.76 ± 0.01	3.47 ± 0.13	0.10 ± 0.00	0.69 ± 0.01
	U SE-MSB (50)	0.30	2.76 ± 0.01	3.47 ± 0.13	0.10 ± 0.00	0.69 ± 0.01
	U BUDDy (100 $\dagger$)	0.71	2.64 ± 0.01	0.79 ± 0.10	0.05 ± 0.00	0.80 ± 0.00
	P Sepformer	0.31	2.80 ± 0.01	5.16 ± 0.08	<u>0.05 ± 0.00</u>	0.79 ± 0.00
	P SGMSE+	0.33	2.78 ± 0.01	4.64 ± 0.09	<u>0.05 ± 0.01</u>	0.80 ± 0.00
	P UNIVERSE++	<u>0.28</u>	<u>3.01 ± 0.01</u>	<u>6.34 ± 0.08</u>	0.09 ± 0.00	<u>0.86 ± 0.00</u>
	Identity	1.06	1.63 ± 0.03	0.00 ± 0.00	0.04 ± 0.00	0.76 ± 0.00
CHiME-6	U SE-MSB (10)	0.48	2.25 ± 0.03	–	–	–
	U SE-MSB (50)	0.46	2.27 ± 0.03	–	–	–
	U BUDDy (100 $\dagger$)	0.45	2.16 ± 0.04	–	–	–
	P Sepformer	0.90	1.19 ± 0.02	–	–	–
	P SGMSE+	0.64	1.76 ± 0.04	–	–	–
	P UNIVERSE++	0.66	1.87 ± 0.03	–	–	–
	Identity	1.09	1.32 ± 0.02	–	–	–

Table 4: Evaluation metrics for the mixed speech enhancement tasks, namely WHAM! noise with reverb, and WHAM! noise with reverb and amplitude clipping, with 95% confidence intervals. The Identity baseline corresponds to unprocessed audio. The number of diffusion steps is denoted in parentheses when relevant, except for $\dagger$, which denotes optimization steps. U = unpaired, P = paired. Best unpaired model is **bold**, and best model across paired and unpaired models is underlined.

supervised Sepformer model on the mixed degradation task. Finally, the few-step sampling results show that the performance of SE-MSB is almost unaffected by the number of sampling steps, while the performance of GFB degrades significantly when using fewer sampling steps. Interestingly, WER and, to a lesser extent, SISDRi are negatively affected when using more sampling SE-MSB, possibly hinting at a perception-distortion trade-off [39]. Because SE-MSB maintains high performance at lower step counts, it further reduces the necessary computational budget required for effective speech enhancement.

6 Conclusion

We proposed an end-to-end unpaired speech enhancement model called SE-MSB, which is a novel combination of Diffusion Schrödinger Bridges and an efficient Mamba Diffusion model architecture operating in the raw waveform domain. We demonstrated the utility of the SE-MSB approach in the audio domain, and in particular speech enhancement, for mapping the distribution of degraded speech to the distribution of clean speech from raw waveforms. SE-MSB performs on par with or outperforms other state-of-the-art methods for unpaired speech enhancement, while being orders of magnitude faster during inference.

We also showed that while SE-MSB is moderately outperformed by paired speech enhancement methods, the performance gap is, for some metrics, negligible, meaning that SE-MSB is a compelling method for adapting speech enhancement models to in-the-wild data without the need for paired examples, thereby improving the overall performance of speech enhancement models in unknown acoustic environments.

This shows renewed promise for efficient and robust speech enhancement in domains where paired data are unavailable or prohibitively expensive to obtain, such as Lombard speech, style transfer, and the reconstruction of historical recordings.

References

- [1] Viraj Kulkarni, Milind Kulkarni, and Aniruddha Pant, “Survey of personalization techniques for federated learning,” 2020.
- [2] “The lombard reflex and its role on human listeners and automatic speech recognizers - pubmed,” [Online; accessed 2026-05-06].
- [3] Jean-Marie Lemercier, Julius Richter, Simon Welker, Eloi Moliner, Vesa Välimäki, and Timo Gerkmann, “Diffusion models for audio restoration,” 2024.
- [4] P. Dhariwal and A. Nichol, “Diffusion models beat GANs on image synthesis,” 2021.
- [5] Z. Kong, W. Ping, J. Huang, K. Zhao, and B. Catanzaro, “Diffwave: A versatile diffusion model for audio synthesis,” 2021.
- [6] J. Richter, S. Welker, J. Lemercier, B. Lay, and T. Gerkmann, “Speech enhancement and dereverberation with diffusion-based generative models,” 2023.
- [7] E. Schrödinger, “Sur la théorie relativiste de l’électron et l’interprétation de la mécanique quantique,” *Annales de l’institut Henri Poincaré*, vol. 2, no. 4, pp. 269–310, 1932.
- [8] Tri Dao and Albert Gu, “Transformers are ssms: Generalized models and efficient algorithms through structured state space duality,” 2024.
- [9] Z. Kong, K. J. Shih, W. Nie, A. Vahdat, S. Lee, J. F. Santos, A. Jukic, R. Valle, and B. Catanzaro, “A2SB: Audio-to-Audio Schrodinger bridges,” 2025.
- [10] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” 2020.
- [11] Maitreya Patel, Mirali Purohit, Jui Shah, and Hemant A. Patil, “Cinc-gan for effective f0 prediction for whisper-to-normal speech conversion,” 2020.
- [12] E. Moliner, S. Braun, and H. Gamper, “Gaussian flow bridges for audio domain transfer with unpaired data,” 2024.
- [13] Carlos Hernandez-Olivan, Koichi Saito, Naoki Murata, Chieh-Hsin Lai, Marco A. Martínez-Ramírez, Wei-Hsiang Liao, and Yuki Mitsufuji, “Vrdmg: Vocal restoration via diffusion posterior sampling with multiple guidance,” 2023.
- [14] Jean-Marie Lemercier, Eloi Moliner, Simon Welker, Vesa Välimäki, and Timo Gerkmann, “Unsupervised blind joint dereverberation and room acoustics estimation with diffusion models,” 2025.
- [15] Eloi Moliner, Jaakko Lehtinen, and Vesa Välimäki, “Solving audio inverse problems with a diffusion model,” 2023.
- [16] V. De Bortoli, I. Korshunova, A. Mnih, and A. Doucet, “Schrodinger bridge flow for unpaired data translation,” in *Advances in Neural Information Processing Systems*, 2024, vol. 37, pp. 103384–103441.
- [17] Yuyang Shi, Valentin De Bortoli, Andrew Campbell, and Arnaud Doucet, “Diffusion schrödinger bridge matching,” 2023.
- [18] Xiangyu Zhang, Qiquan Zhang, Hexin Liu, Tianyi Xiao, Xinyuan Qian, Beena Ahmed, Eliathamby Ambikairajah, Haizhou Li, and Julien Epps, “Mamba in speech: Towards an alternative to self-attention,” 2025.
- [19] Jing Yang, Sirui Wang, Chao Wu, Lei Guo, and Fan Fan, “Schrödinger bridge mamba for one-step speech enhancement,” 2026.
- [20] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” 2022.
- [21] Olaf Ronneberger, Philipp Fischer, and Thomas Brox, “U-net: Convolutional networks for biomedical image segmentation,” 2015.
- [22] Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin, “Understanding and improving layer normalization,” 2019.
- [23] J. Yamagishi, C. Veaux, and K. MacDonald, “CSTR VCTK Corpus: English multi-speaker corpus for CSTR voice cloning toolkit (version 0.92),” 2019.

- [24] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An ASR corpus based on public domain audio books,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015.
- [25] J. Traer and J. H. McDermott, “Statistics of natural reverberation enable perceptual separation of sound and space,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 48, pp. E7856–E7865, 2016.
- [26] I. Szöke, M. Skácel, L. Mošner, J. Paliesek, and J. Černocký, “Building and evaluation of a real room impulse response dataset,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 4, pp. 863–876, 2019.
- [27] D. Murphy and F. Stevens, “Open Air Library 2025,” 2025.
- [28] Real World Computing Partnership, “RWCP (RWCP-SSD),” 2007.
- [29] R. Stewart and M. Sandler, “Database of omnidirectional and b-format impulse responses,” in *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, Dallas, Texas, 2010.
- [30] G. Wichern, J. Antognini, M. Flynn, L. R. Zhu, E. McQuinn, D. Crow, E. Manilow, and J. Le Roux, “Wham!: Extending speech separation to noisy environments,” 2019.
- [31] L. Drude, J. Heymann, C. Boeddeker, and R. Haeb-Umbach, “NARA-WPE: A python package for weighted prediction error dereverberation in numpy and tensorflow for online and offline processing,” in *Speech Communication; 13th ITG-Symposium*, 2018.
- [32] C. Subakan, M. Ravanelli, S. Cornell, M. Bronzi, and J. Zhong, “Attention is all you need in speech separation,” 2021.
- [33] C. K A Reddy, V. Gopal, and R. Cutler, “Dnsmos: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors,” 2021.
- [34] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, “Sdr - half-baked or well done?,” 2018.
- [35] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, “Fréchet audio distance: A metric for evaluating music enhancement algorithms,” 2019.
- [36] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” 2022.
- [37] N. R. Koluguri, T. Park, and B. Ginsburg, “TitaNet: Neural model for speaker representation with 1d depth-wise separable convolutions and global context,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022.
- [38] Y. Wu, K. Chen, T. Zhang, Y. Hui, M. Nezhurina, T. Berg-Kirkpatrick, and S. Dubnov, “Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation,” 2024.
- [39] Yochai Blau and Tomer Michaeli, “The perception-distortion tradeoff,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 2018, p. 6228–6237, IEEE.
- [40] Ilya Loshchilov and Frank Hutter, “Decoupled weight decay regularization,” 2019.
- [41] Jean-Marie Lemerrier, Julius Richter, Simon Welker, and Timo Gerkmann, “Storm: A diffusion-based stochastic regeneration model for speech enhancement and dereverberation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 2724–2737, 2023.
- [42] Lukas Drude, Jahn Heymann, Christoph Boeddeker, and Reinhold Haeb-Umbach, “NARA-WPE: A python package for weighted prediction error dereverberation in Numpy and Tensorflow for online and offline processing,” in *13. ITG Fachtagung Sprachkommunikation (ITG 2018)*, Oct 2018.
- [43] “Csr-i (wsj0) complete - linguistic data consortium,” [Online; accessed 2026-04-30].
- [44] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” 2021.

A Training

We use the AdamW optimizer [40] with a learning rate of 10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.999$, and a weight decay of 0.0. We use a constant learning rate. The models are trained for 250k iterations during pre-training and 100k iterations during fine-tuning. During fine-tuning, we simulate the forward and backward processes using 20 diffusion steps. Data is resampled to 16 kHz and chopped/extended to exactly 1.00 seconds during training, and exactly 5.12 seconds during testing. All audio is normalized to -20 dBFS. After normalization, each audio sample is multiplied by 20 to ensure gradients of a reasonable scale during training, and we use an L1 loss between the model prediction and the target output. Gradients are clipped to a maximum norm of 1.0, and we use mixed precision training (BF16). Models are compiled using PyTorch’s `torch.compile` for faster training. We use a batch size of 8, meaning that the effective batch size is 16 since we train the forward and backward processes simultaneously. We use EMA parameters with a decay of 0.9999 and a warmup period of 89900 steps. Each SE-MSB model is trained on a single NVIDIA GeForce RTX 4090 GPU with 24GB of VRAM for approximately 1 day and 9 hours.

A.1 Dataset descriptions

VCTK

We use the VCTK dataset [23] for clean speech samples during training and evaluation. The dataset consists of 109 English speakers. Each speaker utters approximately 400 sentences for a total of approximately 44 hours of speech data. The audio is recorded at 48 kHz but resampled to 16 kHz during training and evaluation. We choose to use VCTK specifically due to its prominence in the unpaired speech enhancement literature, therefore enabling a fairer comparison to other methods. The dataset is available online at huggingface.co/datasets/badayvedat/VCTK

Room Impulse Responses (RIRs)

For the RIRs, we use a number of different datasets available online [25, 26, 27, 28, 29]. We use the same RIR datasets as the GFB model for a fairer comparison. In total, the RIR dataset consists of approximately 50 minutes of audio data. The collection of all RIR datasets is available online at huggingface.co/datasets/andnymand/RIR-datasets.

WSJ0 Hipster Ambient Mixtures (WHAM!)

For the noise samples, we use the WHAM! dataset [30]. The WHAM! dataset consists of approximately 78 hours of noise samples recorded in various real-world environments. The audio is sampled at 48 kHz but resampled to 16 kHz during training and evaluation. The dataset is available online at huggingface.co/datasets/philgzl/wham.

Libri Speech

We also evaluate SE-MSB and baselines for the dereverberation task on the LibriSpeech ASR corpus, a large-scale corpus of read English speech [24]. Here, we use only the `test-clean` split for evaluation with approximately 5.4 hours of clean speech from 40 different speakers. The dataset is available online at huggingface.co/datasets/openslr/librispeech_asr.

B Model architecture

Mamba Diffusion Model

For model architecture, we use a custom Mamba Diffusion Model built on top of the Mamba2 architecture¹ [8]. The architecture is visualized in figure 3. The model takes as input the raw waveform x_t , the current timestep $t \in [0, 1]$, and a binary conditioning variable c indicating whether the model is training the forward or backward process. The waveform x_t is processed by a 1D convolutional layer with a kernel size of 256, a stride of 16, and a padding of 120. The timestep t is processed by a timestep embedder similar to the one used in [4]. The conditioning variable c is

¹<https://github.com/state-spaces/mamba>

encoded using a learned embedding, one for each direction, and added to the timestep embedding. The "flip" block refers to a flip of the time dimension, and the "concat" block refers to a concatenation along the channel dimension. Each Mamba block uses a model dimension of 512 and a state space dimension of 128. The remaining hyperparameters are the same as the default Mamba2 hyperparameters in the `mamba-ssm` library. Our model uses 10 Mamba Diffusion Blocks, and the total number of parameters is approximately 45 million. The inputs has the following shapes: $x_t \in \mathbb{R}^{B \times C \times T}$, $t \in \mathbb{R}^B$, and $c \in \{0, 1\}^B$, where B is the batch size, C is the number of channels (1 in all our cases), and T is the sequence length. The output of the model has the same shape as the input waveform x_t . The input to the Mamba Diffusion Block, x and c , has shape $x \in \mathbb{R}^{B \times C' \times T'}$ and $c \in \mathbb{R}^B$, where C' and T' are the number of channels and the sequence length after the initial convolutional layer, respectively. For our specific convolutional layer, $C' = 512$ and $T' = T/16$. Here, c is the conditioning variable consisting of the embeddings of the current timesteps t and the indicator variable s , and x is the output of the initial convolutional layer or the output of the previous Mamba Diffusion Block.

C Baselines

Gaussian Flow Bridge (GFB)

We use the GFB implementation provided by the original authors [12]. Their code is available at github.com/microsoft/GFB-audio-control. A Gaussian Flow Bridge is an unsupervised generative model that learns to map two or more distributions to a shared latent space, namely a Gaussian distribution. Then, during inference, the model can map samples to and from the latent space, therefore enabling transformations between the distributions that the model was trained on. We use the pre-trained model checkpoint for reverberant speech enhancement with chunk size $N_c = 128$ available on the GitHub repository. For more info on the chunk size, please refer to their paper and codebase. The GFB model is also trained on clean speech from the VCTK dataset with the same reverberation datasets as our model, also at 16 kHz. The GFB model uses approximately 44 million parameters, which is comparable to our model's 45 million parameters. Importantly, the GFB model is conditioned on so-called reverberant descriptors, namely reverberation time (T_{60}) and clarity (C_{50}). These two descriptors are explicitly provided to the model as a conditioning variable. Our model is not conditioned on any explicit descriptors or any other auxiliary information about the degradation.

Blind Unsupervised Dereverberation with Diffusion Models (BUDDy)

We use the BUDDy implementation provided by the original authors [14]. Their code is available at github.com/sp-uhh/buddy. BUDDy is an unsupervised dereverberation method that uses a pre-trained diffusion model as a prior for the clean speech distribution. It works by jointly estimating the acoustic parameters of the reverberation and the clean speech signal. We use the pre-trained model checkpoint available online. The pre-trained diffusion model uses the NCSN++M model architecture [41] with approximately 27.8 million parameters. The model is also trained on clean speech from the VCTK dataset at 16 kHz. BUDDy is not explicitly trained for dereverberation, but rather the model assumes that the input is clean speech convolved with the acoustic characteristics of the room.

Weighted Prediction Error (WPE)

We use the NARA-WPE implementation provided by [42]. The WPE algorithm models late reverberation as a delayed linear autoregressive process of previously observed signals in the STFT domain. The main assumption is that each current audio frame can be split in two parts, one part for desired early speech and one part for undesired late reverberation. WPE estimates the late reverberant tail by applying a linear prediction filter to past frames. It is a maximum likelihood method that requires no prior knowledge of the room acoustics. We use the following hyperparameters for the WPE algorithm: `taps=20`, `delay=3`, `iterations=5`, `stft_size=512`, `stft_shift=128`, and `statistics_mode="full"`.

Score-based Generative Models for Speech Enhancement (SGMSE)

We use the SGMSE implementation provided by the original authors [6]. Their code is available at github.com/sp-uhh/sgmse. The model is trained on clean speech samples from the WSJ0 dataset

[43], and each clean speech sample is convolved with a simulated room impulse response. SGMSE is trained for paired dereverberation. We use their pre-trained model checkpoint available online. The model uses the NCSN++ model architecture [44] with approximately 65 million parameters.

Paired SE-MSB / Audio-to-Audio Schrödinger Bridge (A²ASB)

For the A²ASB baseline, in the paper called Paired SE-MSB, we adapt a different implementation compared to the original authors [9]. The original A²ASB implementation is trained for bandwidth extension and inpainting on 44 kHz audio using an adapted frequency domain representation. Instead, we train our own A²ASB model for dereverberation using the same architecture, data, and parameters as the unpaired SE-MSB model, see section A. Theoretically, the only difference between SE-MSB and A²ASB is that the latter is trained on paired examples, in addition to the network architecture and data representation. Generally, A²ASB can be seen as the paired version of the SE-MSB model. Formally, A²ASB only trains the backward process, and intermediate samples are drawn from the distribution:

$$\mathbf{X}_{t_k} \sim \mathcal{N}(\mu_t, \Sigma_t)$$

Where

$$\mu_t = \frac{\bar{\sigma}_t^2 + \mathbf{X}_0 \sigma_t^2 \mathbf{X}_1}{\bar{\sigma}_t^2 + \sigma_t^2}, \quad \Sigma_t = \frac{\bar{\sigma}_t^2 \sigma_t^2 \mathbf{I}}{\bar{\sigma}_t^2 + \sigma_t^2}$$

and

$$\sigma_t^2 = \int_0^t \beta_\tau d\tau, \quad \bar{\sigma}_t^2 = \int_t^1 \beta_\tau d\tau$$

Assuming that $\beta_t = \beta$ is constant, we can simplify the above to:

$$\mu_t = (1-t)\mathbf{X}_0 + t\mathbf{X}_1, \quad \Sigma_t = \beta t(1-t)\mathbf{I}$$

This formulation is mathematically equivalent to the DSB Matching intermediate distribution in 3. Similarly, A²ASB uses the following posterior distribution for the backward process:

$$p(\mathbf{X}_{t-\Delta t} | \mathbf{X}_0, \mathbf{X}_t) = \mathcal{N}\left(\frac{(\Delta\sigma_t^2)\mathbf{X}_0 + \sigma_t^2\mathbf{X}_t}{\Delta\sigma_t^2 + \sigma_t^2}, \frac{(\Delta\sigma_t^2)\sigma_t^2}{\Delta\sigma_t^2 + \sigma_t^2}\mathbf{I}\right)$$

Where $\Delta\sigma_t^2 = \sigma_t^2 - \sigma_{t-\Delta t}^2$. This is mathematically equivalent to the posterior distribution in 2 assuming a constant β_t .

Sepformer (Separation Transformer)

We use a SepFormer model implemented with speechbrain available at huggingface.co. The SepFormer is a transformer based neural network for speech separation, that learns short and long-term dependencies through a learned encoded representation and iterative masking procedure done by two Multi-Head Attention blocks called an IntraTransformer and InterTransformer respectively. The model is trained on the *WHAM!* dataset, which we also evaluate on for the mixed speech enhancement tasks. The SepFormer is trained for denoising, and not mixed tasks.

D Computational efficiency

In addition to TFLOPS, we also report inference time in milliseconds for all dereverberation models and the number of trainable parameters in Figure 3. TFLOPS vs Inference time is based on 100 forward passes on samples of 2.048 seconds at 16 kHz using a batch size of 1 and inference mode. Inference time is measured on a consumer-grade GPU and CPU, namely an AMD Ryzen 7 7700 8-Core Processor CPU with 16GB of RAM, and an NVIDIA RTX 4070 Super with 12GB of VRAM. To show that the selective scan of the Mamba2 block has a flop count of $FLOP_{scan} = 2BTN(HQ + 3D)$, we refer to chapter 6 of the original Mamba2 paper [8]. They introduce the notation $BMM(B, M, N, K)$ to define a batched matrix multiplication with a total computation cost of $O(BMKN)$ FLOPs per batch element per head. The $O()$ notation here signifies a multiply and an add function, so we use a factor 2. According to the paper the computational cost arises from a kernel matrix computation with cost $BMM(T/Q, Q, Q, N) = 2 \cdot B \cdot T \cdot Q \cdot N \cdot H$, an input state multiplication with a cost of $BMM(T/Q, Q, P, N) = 2 \cdot B \cdot T \cdot H \cdot P \cdot N$. There are then three low rank block processes, one right factor with a cost of $BMM(T/Q, N, P, Q) = 2 \cdot B \cdot T \cdot H \cdot P \cdot N$,

a left factor with a cost of $BMM(T/Q, Q, P, N)$ with the same cost as the right factor, and a central factor with a cost that is noted negligible by the authors. Summing, we get a total cost of

$$FLOP_{scan} = 2 \cdot B \cdot T \cdot N (Q \cdot H + 3 \cdot H \cdot P) = 2 \cdot B \cdot T \cdot N (Q \cdot H + 3D),$$

where it has been used that $D = H \cdot P$ is the total inner dimension, resulting from a head dimension P times number of heads H .

WPE Flops

For WPE, the STFT-transformed input signal is split into F frequency bins over T time frames. The signal is split into D_c microphone channels and K_t filter taps. The algorithm requires calculating a covariance matrix R_f by an outer product of two $D_c K_t \times 1$ complex vectors for each frequency bin and time frame, requiring $8TF(D_c K_t)^2$ FLOPs. Next, a cross-correlation vector P_f is calculated from a complex $D_c K_t \times 1$ vector, which is multiplied by a complex $D_c \times 1$ vector, requiring $8TFD_c^2 K_t$ FLOPs. These are used to solve a linear system $G_f = R_f^{-1} P_f$, requiring $F \left(\frac{8}{3} (D_c K_t)^3 + 8(D_c K_t)^2 D_c \right)$ FLOPs, and finally this system is applied to the reverberation tail for an additional $8FTD_c^2 K_t$ FLOPs.